

# ExTraCT – Explainable Trajectory Corrections for language-based human-robot interaction using Textual feature descriptions

J-Anne Yow<sup>1,2\*</sup>, Neha Priyadarshini Garg<sup>1</sup>, Manoj Ramanathan<sup>1</sup> and Wei Tech Ang<sup>1,2</sup>

<sup>1</sup>Rehabilitation Research Institute of Singapore (RRIS), 11 Mandalay Road, #14-03 Clinical Science Building, Singapore 308232. RRIS is a joint research institute by Nanyang Technological University (NTU), Agency for Science, Technology and Research (A\*STAR) and National Healthcare Group (NHG), Singapore

<sup>2</sup>Singapore-ETH Centre, Future Health Technologies Programme, CREATE campus, 1 CREATE Way, #06-01 CREATE Tower, Singapore 138602.

Correspondence\*:

J-Anne Yow

janne.yow@ntu.edu.sg

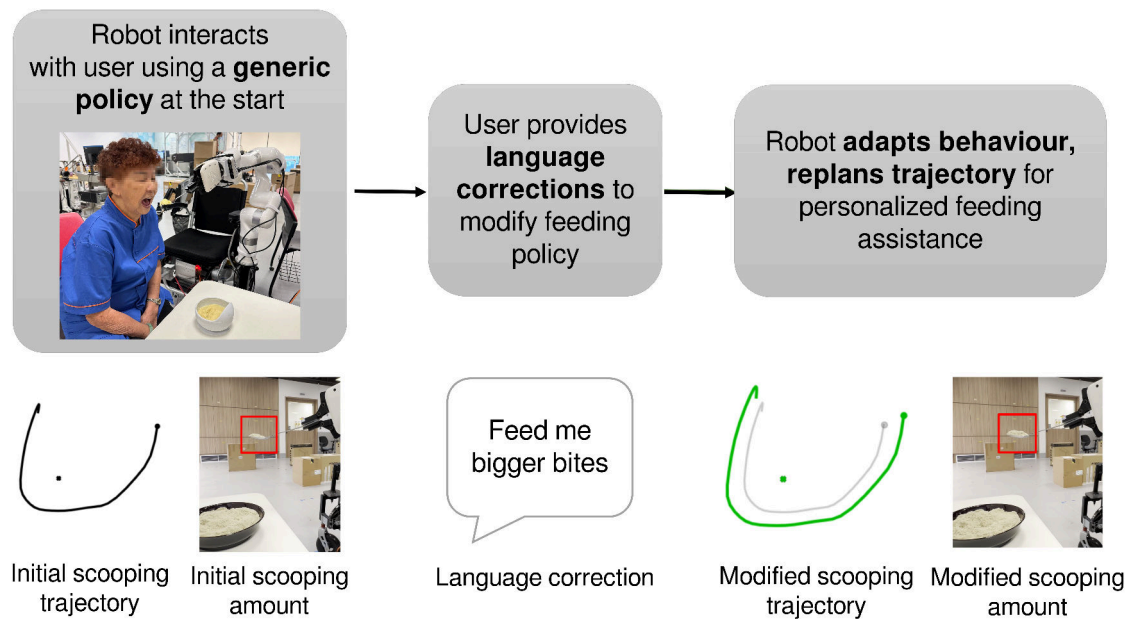
## 2 ABSTRACT

3 In human-robot interaction (HRI), understanding human intent is crucial for robots to perform tasks  
4 that align with user preferences. Traditional methods that aim to modify robot trajectories based on  
5 language corrections often require extensive training to generalize across diverse objects, initial  
6 trajectories, and scenarios. This work presents ExTraCT, a modular framework designed to modify  
7 robot trajectories (and behaviour) using natural language input. Unlike traditional end-to-end  
8 learning approaches, ExTraCT separates language understanding from trajectory modification,  
9 allowing robots to adapt language corrections to new tasks – including those with complex motions  
10 like scooping – as well as various initial trajectories and object configurations without additional  
11 end-to-end training. ExTraCT leverages Large Language Models (LLMs) to semantically match  
12 language corrections to predefined trajectory modification functions, allowing the robot to make  
13 necessary adjustments to its path. This modular approach overcomes the limitations of pre-  
14 trained datasets and offers versatility across various applications. Through comprehensive user  
15 studies conducted in simulation and with a physical robot arm, we demonstrate that ExTraCT's  
16 trajectory corrections are more accurate and preferred in 80% of cases compared to the baseline.  
17 Additionally, we demonstrate the adaptability and effectiveness of our framework in a complex  
18 scenario like assistive feeding.

19 **Keywords:** human-robot interaction, language in robotics, natural language processing, assistive robots, foundational models, LLMs

## 1 INTRODUCTION

20 Motion planning algorithms optimize trajectories based on predefined cost functions, which usually  
21 consider robot dynamics and environmental constraints. However, robots need to account for human  
22 preferences to assist effectively when working with humans. Human preferences can vary based on the  
23 environment and human factors; e.g., during mealtimes, users may prefer the robot to feed bigger bites

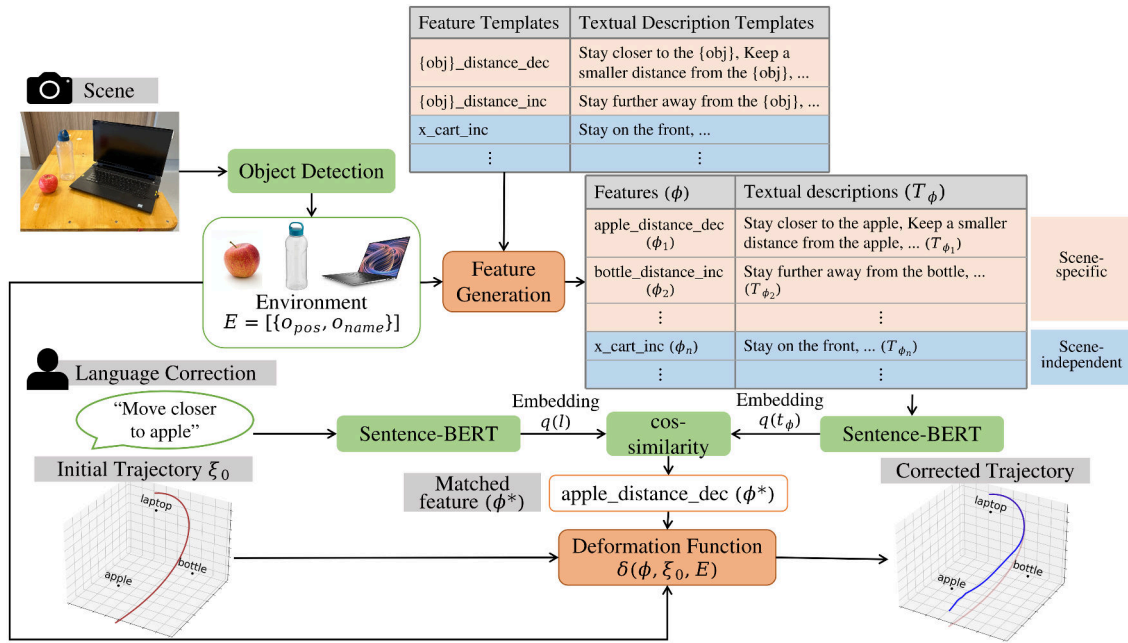


**Figure 1.** Human-robot interaction during assistive feeding. When interacting with the user for the first time, the robot executes a generic policy. Given user feedback in the form of language, the robot then modifies its behaviour to personalize its assistance.

when hungry but may prefer the robot to feed smaller bites when full (Fig. 1); when throwing trash, some may prefer the robot to place the trash away from food items to maintain hygiene, while others may prefer it to take the shortest route for efficiency. Thus, incorporating all possible human preferences in the cost function is challenging. To address this, we explore the problem of natural language trajectory corrections, i.e., modifying a robot's initial trajectory based on natural language corrections provided by a human. We choose natural language as it provides an intuitive and expressive way for humans to convey their preferences.

A key challenge in natural language trajectory corrections is mapping the natural language to the robot action, which is the deformed trajectory. Existing works (Broad et al., 2017; Sharma et al., 2022; Bucker et al., 2022, 2023) try to learn a direct mapping between natural language and robot trajectories or actions using offline training paradigms. To improve generalization to different objects and phrases, some works (Sharma et al., 2022; Bucker et al., 2022, 2023) leverage on foundational models, i.e., BERT (Devlin et al., 2018) and CLIP (Radford et al., 2021). However, these models struggle to generalize to varied initial trajectories and object poses due to dataset limitations. Furthermore, with a direct mapping learnt between natural language and robot action, it is difficult to understand the root cause of failures Adadi and Berrada (2018), which we believe could stem from issues in language grounding, scene understanding, or inaccuracies in trajectory deformation function.

To address these limitations, we separate language understanding from trajectory deformation, thus enabling a more accurate interpretation of instructions. First, we match the language corrections to a short description of the change in trajectory, which we term a feature. We define a set of feature templates and their corresponding textual description templates, which are then used to generate features and their textual descriptions (Fig. 2) for any given scene online. The language uttered by the user can then be mapped to the most likely feature by computing the semantic similarity between the textual descriptions of the feature and the correction uttered by the user using Large Language Models (Devlin et al., 2018; Wang et al., 2020;



**Figure 2.** Architecture of ExTraCT. Given the objects in a scene, the features  $\phi$  and corresponding textual descriptions  $T_\phi$  are generated online. We obtain the embeddings of the language correction  $q(l)$  and the phrases ( $t_\phi \in T_\phi$ ) in the textual descriptions of the features  $q(t_\phi)$ , and use semantic textual similarity to obtain the most similar textual description, which is mapped to feature  $\phi^*$ . A deformation function  $\delta$  is used to deform the initial trajectory  $\xi_0$  based on the feature  $\phi^*$  and the object positions in the environment  $E$ . A trajectory optimizer is used to ensure that the robot's kinematic constraints are satisfied.

48 Brown et al., 2020; Chowdhery et al., 2022). For features that are mapped with insufficient confidence, our  
 49 approach can alert the end-user instead of generating a random modified trajectory. In this work, we set the  
 50 confidence threshold to be 0.6, which was determined empirically based on a small set of testing data. Note  
 51 that this test data is different from the set we used in the experiments below. If the threshold is set too high,  
 52 the system would alert the user too frequently that feature matching failed. Conversely, a threshold too low  
 53 could lead to incorrect feature matching, i.e. matching to a feature even though the correction could be out  
 54 of the feature space.

55 Once the most likely feature is determined, the initial trajectory can be modified based on a trajectory  
 56 deformation function, allowing generalization to different object configurations and trajectories.

57 This separation of language understanding from trajectory deformation allows for more precise and  
 58 context-aware robotic responses. By decoupling these two elements, our method can easily expand to new  
 59 tasks, as shown in Section 4.5. Furthermore, it provides clearer insights into potential sources of failure,  
 60 as it distinguishes between errors in language interpretation and trajectory execution. Thus, our approach  
 61 improves the accuracy and generalization of language corrections and enhances the interpretability and  
 62 reliability of robotic systems in executing these corrections.

63 While our approach is limited by the feature types in the feature templates, end-to-end training methods  
 64 would encounter similar limitations when dealing with unseen features. However, they are additionally  
 65 limited by the object configurations and initial trajectories used in the training data, as shown in Section 4.1.  
 66 Our formulation can generalize to different object configurations and initial trajectories without training  
 67 a model. Our key insight is that integrating the strengths of LLMs in handling language diversity with  
 68 a hand-crafted approach for trajectory modifications bypasses the need for end-to-end training while

69 achieving comparable or better performance. Furthermore, end-to-end training methods are data-intensive.  
70 Collecting data is challenging in the robotics domain, while generating data would require a similar amount  
71 of hand-crafted features as our proposed approach.

72 We evaluated our system through within-subject user studies in simulation and the real world. Our  
73 results show that our method had higher accuracy and was rated higher in approximately 80% of test cases  
74 compared to the state-of-the-art method LaTTe (Bucker et al., 2023), which also uses LLMs but was trained  
75 in an end-to-end fashion. We further analysed the failure cases of our method and showed that our method  
76 could be improved further by adding more phrases to the textual description templates.

77 Our contributions in this work are threefold: (1) We introduce a modular framework that integrates LLMs  
78 with trajectory deformation functions for trajectory corrections using language without end-to-end training.  
79 (2) We conducted extensive quantitative experiments on a substantial dataset, complemented by user studies  
80 comparing our method against a baseline that deforms trajectories based on language in an end-to-end  
81 fashion. (3) We demonstrate the versatility of our framework through its application to a range of tasks,  
82 including general object manipulation and assistive feeding.

## 2 RELATED WORK

83 There are various ways of conveying human preference to the robot for correcting a trajectory, such as  
84 through language (Bucker et al., 2022, 2023; Sharma et al., 2022), physical interaction (Bajcsy et al., 2017,  
85 2018; Bobu et al., 2021), rankings (Jain et al., 2015) and joystick inputs (Spencer et al., 2020). In this work,  
86 we use language as it is the most natural and intuitive way of communicating human preference (Tellex  
87 et al., 2020).

88 Language correction works can be broadly classified into two categories – generating new trajectories  
89 (Broad et al., 2017; Sharma et al., 2022; Liang et al., 2023; Zha et al., 2023) and modifying existing  
90 ones (Bucker et al., 2022, 2023). The first category, trajectory generation, involves creating new motion  
91 plans to enable robots to correct errors to complete tasks. Consider a task where a robot provides feeding  
92 assistance to a user. An online correction such as “move 5cm to the left” when acquiring the food might be  
93 provided so the robot can align more accurately with the food morsel before acquiring it. This directs the  
94 robot to generate a new trajectory to move to the left for more precise alignment. On the other hand, our  
95 work focuses on the second category of corrections, which involves modifying an existing trajectory. This  
96 includes relative corrections such as “scoop a larger amount of food”. Such corrections require adjustments  
97 to an existing trajectory, requiring an understanding of the initial plan or trajectory. Modifying existing  
98 trajectories is important, as human preferences are commonly expressed in relative terms.

99 Both categories of language corrections face a common challenge: translating natural language to robot  
100 actions, also known as language grounding. Language grounding can be categorized into three approaches –  
101 semantic parsing to probabilistic graphs, end-to-end learning using embeddings and prompting LLMs to  
102 generate code.

### 103 2.1 Semantic Parsing to Probabilistic Graphs

104 Earlier works in language correction use a grammatical structure to represent language and ground  
105 language by learning the weights of functions of factors (Broad et al., 2017). A probabilistic graphical  
106 model (distributed correspondence graph, DCG) is used to ground language to a set of features that relate  
107 to the environment and context. Each feature represents a cost or constraint, which a motion planner then  
108 optimizes. Given the variety of possible phrases a human can provide to correct the same feature, grounding

language using a fixed grammatical structure limits the generalization of this approach. We differ from this approach in how we ground language. Instead of parsing language corrections to phrases with a fixed grammar, we leverage the textual embeddings in LLMs (Devlin et al., 2018; Wang et al., 2020; Brown et al., 2020; Chowdhery et al., 2022) to ground them to textual descriptions of features.

## 2.2 End-to-End Learning Using Embeddings

Prior works have explored LLMs, combining BERT (Devlin et al., 2018) (textual) embeddings and CLIP (Radford et al., 2021) (textual and visual) embeddings to align visual and language representations. Sharma *et al.* (Sharma et al., 2022) learn a 2D cost map from CLIP and BERT embeddings, which converts language corrections to a cost map that is optimized using a motion planner to obtain the corrected trajectory. Bucker *et al.* goes a step further by directly learning a corrected trajectory from CLIP and BERT embeddings in 2D (Bucker et al., 2022) and 3D (Bucker et al., 2023). Additionally, a transformer encoder obtains geometric embeddings of the object poses and an initial trajectory. A transformer decoder combines textual, visual and geometric embeddings to generate the corrected trajectory. Geometric embeddings have also been combined with textual embeddings to learn a robot policy conditioned on language (Cui et al., 2023), but a shared autonomy paradigm was used to reduce the complexity. These approaches deploy an end-to-end learning approach requiring large multi-modal datasets, including image, text, and trajectory data, which are difficult to obtain. Compared to these works, our proposed approach leverages LLMs to summarize natural language corrections into concise and informative textual representations, thus removing the need for multi-modal training data to ground language corrections.

## 2.3 Prompting LLMs to Generate Code

Concurrent with our work, advances in Large Language Models (LLMs) have led to recent works (Liang et al., 2023; Huang et al., 2023; Yu et al., 2023; Zha et al., 2023) employing LLMs to generate executable code from natural language instructions. These works mainly focus on generating robot plans or trajectories given a phrase, either by calling pre-defined motion primitives (Liang et al., 2023; Zha et al., 2023) or by designing specific reward functions (Huang et al., 2023; Yu et al., 2023). This approach has shown versatility in handling diverse instructions and constructing sequential policy logic. However, the adoption of these methods is hindered by significant computational costs due to the need for larger, general-purpose models like GPT-4, which cannot be run on standard commercial hardware. Remote execution via API calls is a viable option, but this is limited by the need for a stable internet connection and the rate limits imposed on the APIs, which can hinder real-time applications. Moreover, similar to our approach that integrates hand-crafted templates, the effective use of LLMs for specific task-oriented code generation also requires many in-context examples. In contrast, our approach provides a more cost-effective solution for language understanding in robotic systems by using semantic textual similarity, eliminating the need for extensive prompting or high computational demands. Although our method requires additional modules to process complex language utterances, such as referring expressions and compound sentences, it is a more viable alternative in settings constrained by limited computational resources and internet connectivity. This makes our framework well-suited for practical applications where efficiency is a key consideration.

# 3 APPROACH

## 3.1 Problem Definition

Our goal is to develop an interface that allows users to modify the trajectory of robot manipulators based on their preferences conveyed through natural language corrections. More specifically, the problem can



be described as finding the most likely trajectory  $\xi^*$  given the environment  $E$ , the language correction  $l$  provided by the human and an initial trajectory  $\xi_0$ , where  $P(\xi|E, l, \xi_0)$  is the probability of a corrected trajectory given the environment, language correction and initial trajectory.

$$\xi^* = \underset{\xi}{\operatorname{argmax}} P(\xi|E, l, \xi_0) \quad (1)$$

The environment consists of a set of objects, with each object having two attributes – object name  $o_{name}$  and object position  $o_{pos}$ , i.e.,  $E = \{(o_{name}^i, o_{pos}^i)\}$ , where  $i$  is the object index. The object names and poses in the environment can be obtained using perception algorithms (e.g., Mask R-CNN (He et al., 2017), YOLO (Redmon and Farhadi, 2018)) or foundational models (e.g., OWL-ViT (Minderer et al., 2022), Grounding DINO (Liu et al., 2023)). In our work, we used Mask R-CNN to get the environment  $E$ .

### 3.2 Features

The space of possible trajectories  $\xi$  is infinite. However, realistically, it can be bounded by the environment and motion planner constraints (Broad et al., 2017). Therefore, we assume that the language correction  $l$  can be mapped to a finite set of features  $\Phi$  that can be scene-specific or scene-independent. Each feature  $\phi \in \Phi$  corresponds to a deformed trajectory, i.e.,  $\xi = \delta(\phi, \xi_0, E)$  where  $\delta$  is the trajectory deformation function which we describe in Section 3.4. Thus, we can obtain the most likely trajectory  $\xi^* = \delta(\phi^*, \xi_0, E)$  from the most likely feature  $\phi^*$ . This reduces the problem of finding the most likely trajectory  $\xi^*$  to the problem of finding the most likely feature  $\phi^*$ . Thus, our problem can be rewritten as:

$$\phi^* = \underset{\phi \in \Phi}{\operatorname{argmax}} P(\phi|l) \quad (2)$$

where  $P(\phi|l)$  is the probability of feature  $\phi$  given language utterance  $l$ . The features can be categorized into two types – scene-specific and scene-independent features. Scene-specific features depend on the objects in the scene, while scene-independent features are not.

As proof of concept, we define 2 scene-specific and 6 scene-independent feature templates (Table 1). Scene-specific features are generated online for each object detected in the scene, allowing the approach to generalize to different objects. They are defined based on *object distance*, which is to either increase (`obj_distance_increase`) or decrease (`obj_distance_decrease`) the distance to an object. Scene-independent features are based on the 3 axes in *cartesian* space, i.e., to move the gripper up (`z_cart_increase`), down (`z_cart_decrease`), left (`y_cart_decrease`), right (`y_cart_increase`), forward (`x_cart_increase`) and backward (`x_cart_decrease`). The space of feature templates is expandable and can be made specific to the robot's task, as demonstrated in Section 4.5.

### 3.3 Textual Descriptions and Optimal Feature Selection

To compute  $P(\phi|l)$ , we generate a textual description  $T_\phi$  for each feature  $\phi$ . Each textual description consists of a set of language phrases, which includes commonly used phrases to modify the trajectory for a

<sup>1</sup> We assume that a given language correction will correspond to only one feature. Practically, a complex language correction can map to multiple features, but the focus of this work is on comparing our proposed approach against an end-to-end method, which could only handle one feature at a time. We will address this issue in the future.

**Table 1.** Feature Templates (FT) and their Textual Description Templates (TDT)

|     |                                                                                                                                       |                                                                                                                                                                 |
|-----|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FT  | obj_distance_decrease                                                                                                                 | obj_distance_increase                                                                                                                                           |
| TDT | Move closer to {obj}<br><b>Stay close to {obj}</b><br><b>Decrease distance to {obj}</b><br>Keep a smaller distance from {obj}         | Move further away from {obj}<br><b>Stay away from {obj}</b><br><b>Increase distance to {obj}</b><br>Keep a bigger distance from {obj}<br>Avoid {obj}            |
| FT  | z_cart_decrease                                                                                                                       | z_cart_increase                                                                                                                                                 |
| TDT | Move closer to table<br><b>Stay closer to table</b><br>Move lower<br>Move down<br><b>Stay down</b><br>Go to the bottom<br>Down<br>Low | Move further away from table<br><b>Stay away from table</b><br>Move higher<br>Move up<br><b>Stay up</b><br>Stay on the upper part<br>Up<br>Top<br>Go to the top |
| FT  | y_cart_decrease                                                                                                                       | y_cart_increase                                                                                                                                                 |
| TDT | <b>Stay on the left</b><br><b>Go to the left</b><br>Move left<br>Move more towards the left<br>Left                                   | <b>Stay on the right</b><br><b>Go to the right</b><br>Move right<br>Move more towards the right<br>Right                                                        |
| FT  | x_cart_decrease                                                                                                                       | x_cart_increase                                                                                                                                                 |
| TDT | <b>Stay at the back</b><br><b>Go to the back</b><br>Move back<br>Stay back<br>Move backward<br>Go behind                              | <b>Stay at the front</b><br><b>Go to the front</b><br>Move front<br>Stay front<br>Move forward                                                                  |

Note: Only the bolded phrases were used for the analysis in Section 4.3.3.

particular feature. Table 1 shows the textual description templates (TDT) for each feature template (FT). During feature generation for a scene, the {obj} placeholder in scene-specific feature templates will be replaced by the object name  $o_{name}$  obtained using object detection, as shown in Fig. 2.

We initially defined a few phrases in the textual description templates. If feature matching fails due to out-of-distribution user utterances, additional phrases can be added to the textual description templates easily to improve feature matching. We examine how the phrases in the textual description templates can affect feature matching and performance in Section 4.3.3.

Since there is a one-to-one mapping between the feature and its textual description, Eq. 2 can be rewritten as:

$$\phi^* = \operatorname{argmax}_{\phi \in \Phi} P(T_\phi | l) \quad (3)$$

To compute  $P(T_\phi|l)$ , we leverage large language models (LLMs) to map diverse language phrases to fixed-length vectors called embeddings. To capture the semantic meaning of sentences, we used Sentence Transformers (Reimers and Gurevych, 2019), which is fine-tuned for semantic similarity tasks. We chose the pre-trained `all-MiniLM-L6-v2` model (Wang et al., 2020) provided by Sentence Transformers<sup>2</sup> as it provided us with the best trade-off between speed and performance. Semantically closer language phrases are more likely to have higher cosine similarity between their embeddings. Thus,  $P(T_\phi|l)$  can be defined as follows:

$$P(T_\phi|l) \propto \max_{t_\phi \in T_\phi} q(t_\phi) \cdot q(l) / \|q(t_\phi)\| \cdot \|q(l)\| \quad (4)$$

where  $q(x)$  is the embedding for a language phrase  $x$ . Once the most likely feature  $\phi^*$  is obtained, the trajectory  $\xi^*$  can be obtained using a deformation function.

### 3.4 Deformation Function

A deformation function  $\delta(\phi, \xi_0, E)$  modifies a trajectory based on a feature  $\phi$ . First, we calculate the force  $F$  to be exerted on each waypoint of the trajectory.

For scene-specific features, i.e., *object distance* features, the force exerted is dependent on the object position  $o_{pos}$  and a radius of deformation  $r$ . In our experiments, we set  $r = 0.3$ , which was determined empirically. We note that  $r$  can be set adaptively based on environmental constraints and human preferences, but this will be part of our future work. For waypoints within the radius of deformation  $r$  from the object position  $o_{pos}$ , a force is applied on the waypoints in the direction of the distance vector between the waypoint and the object. The force is 0 for other waypoints.

For scene-independent features, a force is exerted on all waypoints of the trajectory, where the direction of the force is dependent on the feature.

The trajectory is deformed based on the force calculated on each waypoint:  $\delta = \xi_0 + wF$ , where the weight  $w$  changes the magnitude of the deformation. We empirically determined the value of  $w$  to be a constant of 1.0 in our experiments, but this can be modified based on the intensity of the language correction in the future.

Finally, the deformed trajectory is passed to a trajectory optimizer to ensure the robot's kinematic constraints are satisfied.

## 4 EXPERIMENTS

We conducted simulation and real-world experiments to validate our proposed approach. We hypothesize that:

**H1** Our approach will be able to generalize to natural language phrases and environments with different objects;

**H2** Our approach will deform trajectories at least as accurately as end-to-end methods for trajectory deformation;

**H3** Our approach will obtain higher rankings from users compared to end-to-end methods;

<sup>2</sup> [https://www.sbert.net/docs/pretrained\\_models.html](https://www.sbert.net/docs/pretrained_models.html)



222 **H4** Our approach will be more interpretable than end-to-end methods.

223 We compared our approach against LaTTe (Bucker et al., 2023), an end-to-end approach for language  
224 corrections that can deform trajectories in 3D space. Experiments were conducted to evaluate the  
225 generalization ability of the proposed approach. We also conducted user studies to evaluate the end-  
226 users' satisfaction with the deformed trajectories and to obtain more diverse language utterances. The  
227 Nanyang Technological University Institutional Review Board approved the user studies. In the next  
228 section, we briefly describe our baseline method, LaTTe.

## 229 4.1 Baseline

230 LaTTe first created a dataset consisting of tuples of initial trajectory, language correction, object images,  
231 names and poses in the scene, and deformed trajectory. This dataset is then used to train a neural network  
232 which can map the initial trajectory, language correction and objects in the scene to a deformed trajectory.  
233 For generalization to various objects in the scene and different natural language phrases, pre-trained BERT  
234 and CLIP are used to generate embeddings for the language correction and the object images, which are  
235 provided as input to the neural network. We used the model provided by the authors<sup>3</sup>, trained on 70k  
236 samples for our experiments. Before generating the deformed trajectory using the trained model, a locality  
237 factor hyper-parameter must be set, determining the range of desired change over the trajectory. In our user  
238 studies, we set the locality factor hyper-parameter to be 0.3, the mean value in their dataset.

## 239 4.2 Generalization Experiments

240 To evaluate the ability of our approach to generalize to different object configurations, trajectories and  
241 corrections, we used the dataset that was originally employed to train the LaTTe. The LaTTe dataset  
242 includes 100k samples, with object names sampled from the Imagenet dataset. The full dataset contains  
243 three types of trajectory modifications – *cartesian* changes, *object distance* changes and *speed* changes.  
244 We removed the samples related to speed changes in our evaluation as we did not include speed features in  
245 our templates, resulting in 65261 samples. We set the locality hyper-parameter for LaTTe by referencing  
246 the value provided in each sample in the dataset.

### 247 4.2.1 Evaluation Metrics

248 To evaluate the accuracy of the deformed trajectories, LaTTe compared the similarity between the  
249 deformed trajectory using their method and the ground-truth output trajectory in the dataset using metrics  
250 like dynamic time warping (DTW) distance. However, this may not accurately capture the correctness of  
251 the trajectory modification. For example, even if the trajectory change occurs in a direction contrary to the  
252 intended one, the DTW distance may still register as small. Additionally, relying on a single ground-truth  
253 trajectory for comparison is problematic, as multiple valid trajectories can achieve the correct deformation.

254 Thus, we developed a way to assess the performance of trajectory deformations based on language. Our  
255 approach evaluates the accuracy of deformed trajectories by considering the type of correction applied, as  
256 shown in Fig. 3.

257 For corrections involving *cartesian* changes, we first sampled a range of trajectory deformations of  
258 different intensities, i.e., by varying the value of  $w$  in the deformation function (Section 3.4) from -2.5  
259 to 2.5. Dynamic time warping (DTW) distance was then used to find the weight that best represents  
260 the deformed trajectory. A deformation is deemed correct if the weight aligns with the intended change

---

<sup>3</sup> <https://github.com/arthurfenderbucker/LaTTe-Language-Trajectory-Transformer>

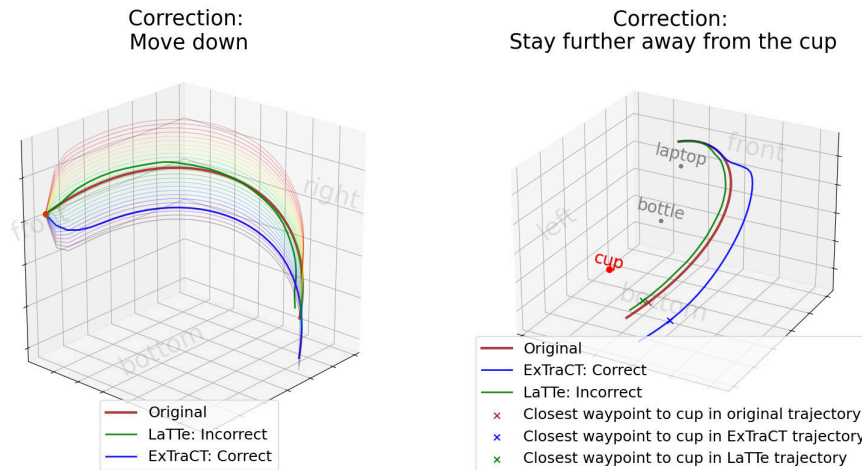


Figure 3a.

Figure 3b.

**Figure 3.** Accuracy evaluation. The green trajectory shows an incorrect deformation, while the blue trajectory shows a correct deformation. **(A)** *Cartesian* changes – we sampled trajectory deformations with varying weights, which affect the intensity of deformation. The sampled trajectories below the original trajectory have negative weights, while those above the original trajectory have positive weights. **(B)** *Object distance* changes – we obtained the waypoints in the original and deformed trajectories and compared the change in distance relative to the target object.

**Table 2.** Generalization Experiment Results

| Approach | Overall       |               | Object Distance Changes |               | Cartesian Changes |               |
|----------|---------------|---------------|-------------------------|---------------|-------------------|---------------|
|          | Accuracy      | DTW           | Accuracy                | DTW           | Accuracy          | DTW           |
| ExTraCT  | <b>89.23%</b> | <b>2.6386</b> | <b>86.10%</b>           | <b>3.0600</b> | 92.27%            | <b>2.2100</b> |
| LaTTe    | 73.37%        | 3.4099        | 53.48%                  | 3.7499        | <b>92.64%</b>     | 3.0804        |

direction. For example, a positive weight should correspond to a positive change (i.e., increase), and a negative weight should correspond to a negative change (i.e., decrease).

For corrections involving *object distance* changes, we measure the accuracy by comparing the distance between the object and the waypoint closest to the object in both the original and deformed trajectories. The key metric is whether the modified trajectory brings the waypoint closer to or further from the target object, which can be quantified without sampling multiple trajectories. A deformation is deemed correct if, for a positive change, the waypoint in the deformed trajectory is further from the object, and for a negative change, it is closer to the object.

To facilitate a direct comparison with the findings reported in LaTTe, we also evaluated the performance in terms of the similarity between the deformed trajectory of each approach and the ground-truth output trajectory in the dataset using the dynamic time-warping (DTW) distance. While LaTTe reported both the DTW distance and the discrete Frechet distance (DFD), we opted for the DTW distance as it provides a more comprehensive similarity measure, accounting for variations in the overall shape of the trajectory and the timing of specific waypoints.

## 4.2.2 Results

Table 2 presents the evaluation results, where our approach outperformed the baseline, showing support for **H1** and **H2**. To determine whether the type of change affects the performance, we analysed the results

stay closer to the Egyptian cat

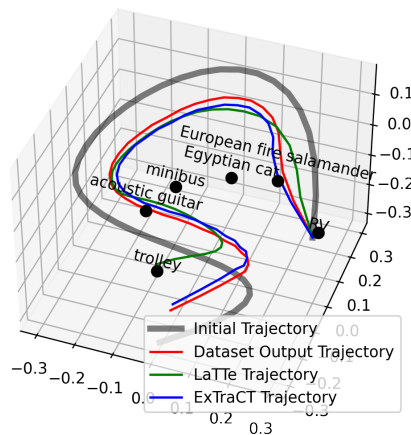


Figure 3a.

stay closer to the Egyptian cat

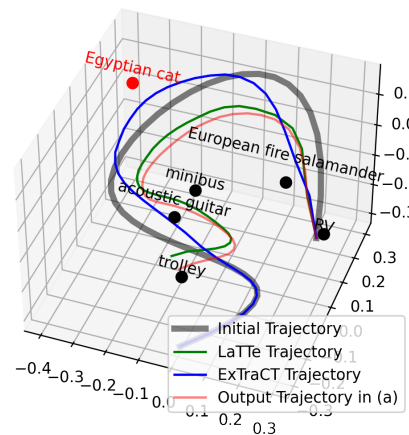


Figure 3b.

Stay further away from the Egyptian cat

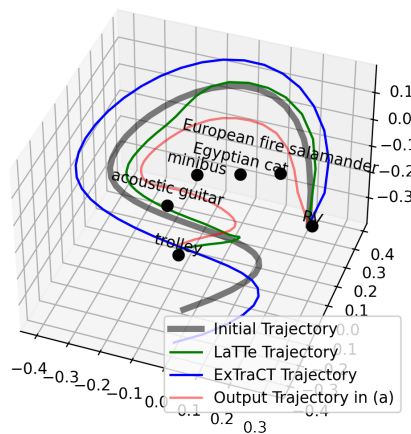


Figure 3c.

stay closer to the Egyptian cat

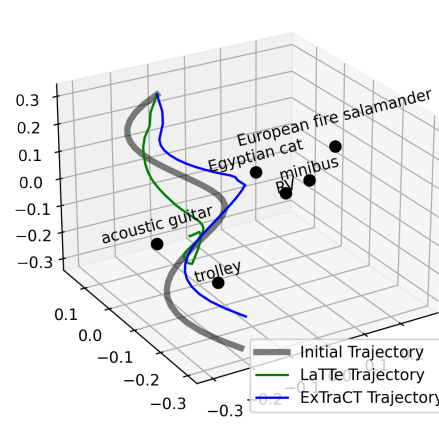


Figure 3d.

**Figure 3.** Changes in the deformed trajectory using (A) a sample in LaTTe's dataset (B) a change in the target object pose (C) a change in the language correction that conveys an opposite meaning (D) a change in the initial trajectory. The deformed trajectory by LaTTe is inaccurate for (B), (C) and (D), while ExTraCT produces a correct trajectory deformation for all cases.

based on the change category, i.e., *object distance* changes and *cartesian* changes. Even though both approaches have comparable performance for *cartesian* changes, we see that the performance of LaTTe degrades for *object distance* changes, with an accuracy of only 54.5%, which is approximately equivalent to random chance.

#### 4.2.3 Analysis of Failure Cases in LaTTe

To better understand the degraded performance of LaTTe for *object distance* changes, we performed qualitative analysis on LaTTe by picking a random sample in LaTTe's dataset (Fig. 3a) and modifying the target object pose (highlighted in red, Fig. 3b), language correction (Fig. 3c) and initial trajectory (Fig. 3d). The trajectories deformed by LaTTe shown in Fig. 3 demonstrate that these changes did not result

287 in an expected change in the deformed trajectory. Fig. 3b and 3c show a similar (but incorrect) deformed  
288 trajectory to the original sample, while Fig. 3d shows the deformed trajectory in the opposite direction to  
289 the correction. On the other hand, ExTraCT deforms the trajectories correctly in all these cases.

290 To understand why a change in the language correction (Fig. 3c) did not result in a correct change  
291 in the deformed trajectory, we analysed the embeddings of the language correction. We used Sentence  
292 Transformers (Reimers and Gurevych, 2019) to find the top eight sentences with the closest semantic  
293 similarity in LaTTe’s dataset. Sentences that convey opposite meanings but have high lexical similarity,  
294 such as “stay closer to the Egyptian cat” had a high cosine similarity score to “stay further away from  
295 the Egyptian cat”, as highlighted in Table 3. This makes learning trajectory deformations from textual  
296 embeddings difficult, as the BERT embeddings used may not always capture the semantic meaning of the  
297 corrections.

298 Unfortunately, for the cases where we modified the target object pose (Fig. 3b) and the input trajectory  
299 (Fig. 3d), it was difficult to understand why failures occurred. Failures could arise from various factors, such  
300 as errors in embedding geometrical information like trajectories and object configurations and insufficient  
301 training data. The lack of transparency makes identifying and rectifying specific issues difficult, motivating  
302 our separation of the problem into two distinct phases – language understanding and trajectory deformation.

**Table 3.** Top 8 Sentences with Highest Similarity Scores for ”Stay further away from the Egyptian cat” in LaTTe’s Dataset

| Sentence                                      | Similarity Score |
|-----------------------------------------------|------------------|
| Stay further away from the Egyptian cat       | 1.00             |
| Stay a lot further away from the Egyptian cat | 0.98             |
| <b>Stay closer to the Egyptian cat</b>        | <b>0.91</b>      |
| Walk a lot further away from the Egyptian cat | 0.90             |
| Walk a bit further away from the Egyptian cat | 0.89             |
| <b>Stay a lot closer to the Egyptian cat</b>  | <b>0.88</b>      |
| <b>Stay very closer to the Egyptian cat</b>   | <b>0.88</b>      |
| <b>Stay a bit closer to the Egyptian cat</b>  | <b>0.87</b>      |

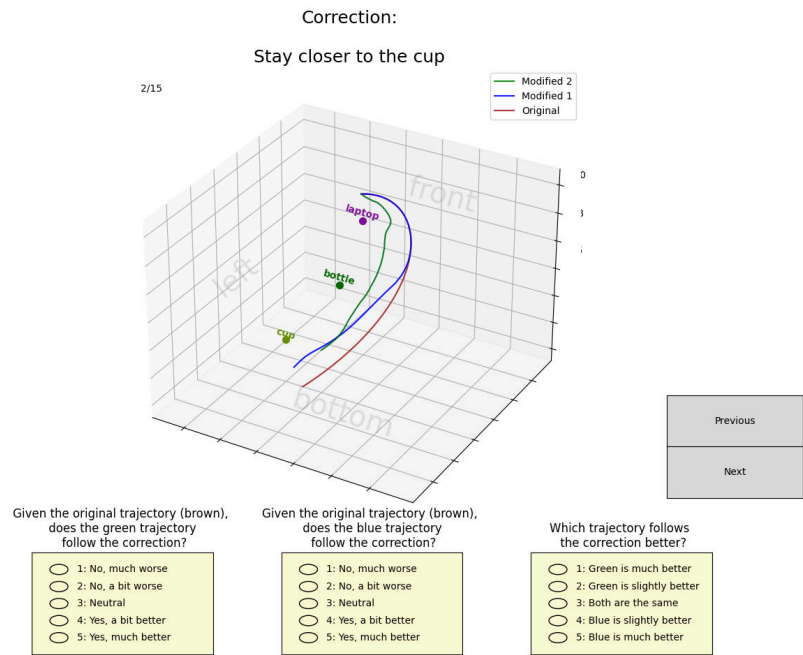
Note: Bolded phrases indicate sentences with opposite meanings.

303 4.2.4 Analysis of Failure Cases in ExTraCT

304 All the failure cases in ExTraCT can be attributed to incorrect feature mapping. For example, “Go  
305 to the upper part” was incorrectly mapped to `z_cartesian_decrease` as the most similar phrase  
306 was “Go to the bottom”. “Drive a lot closer to the meat market” was incorrectly mapped to `meat`  
307 `market_distance_increase` as the closest phrase was “Stay a lot further away from meat market”.  
308 These errors occurred as the embeddings from large language models (LLMs) may not always capture the  
309 nuanced semantic meanings of language phrases (Table 3).

310 To improve the language understanding capabilities of our system, we can either fine-tune the existing  
311 language model for our application or employ a larger LLM with better semantic understanding. Another  
312 way to improve performance is to include previously mismatched phrases in our textual description  
313 templates. In Section 4.3.3, we show how expanding the textual description templates can improve  
314 performance. Note that for this evaluation, we deliberately did not include all the phrases in LaTTe’s dataset  
315 in our template descriptions, as that would naturally lead to an exact match in the sentence, which would  
316 not realistically reflect the true language understanding capabilities of our approach.

317 4.3 User Studies in Simulation



**Figure 4.** Interface for the simulation study showing scene 1. The modified trajectories are displayed on the interface simultaneously for better comparison. The green modified trajectory is by LaTTe, while the blue modified trajectory is by our approach.

318 We recruited 15 subjects (8 male, 7 female) for the simulation study. Out of these participants, 6  
319 individuals did not have a background in robotics. The study interface (Fig. 4) (modified from LaTTe),  
320 displayed the initial trajectory, the modified trajectory using LaTTe and the modified trajectory using our  
321 method (ExTraCT) at the same time. The trajectories were displayed to the users in 3D and subjects could  
322 interact with the plots to change the view. The participants were not informed which method was used  
323 to deform the trajectories, and the modified trajectories were labelled only with "1" and "2". The labels  
324 were kept consistent throughout the experiment so that we could obtain the subjects' overall rankings and  
325 preferences for each method across different scenes.

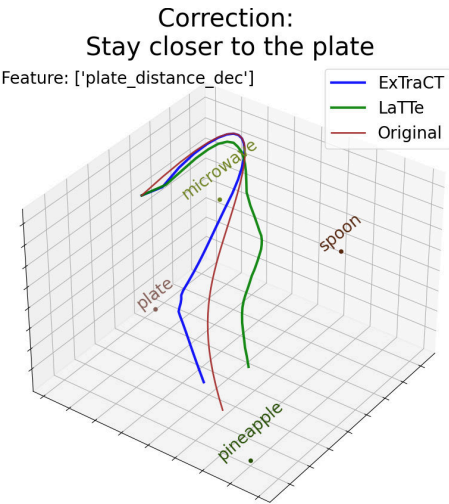
326 3 scenes were presented to each subject, as shown in Figures 4 and 5. Different household objects were  
327 placed at randomized locations for each scene. The objects in scenes 1 and 2 were selected from LaTTe's  
328 dataset, while those in scene 3 were randomly selected and out of LaTTe's dataset. Since there were no  
329 images of the objects, no CLIP image embeddings were used for LaTTe. Instead, CLIP textual embeddings  
330 of the object names were used for LaTTe to identify the correct target object.

331 There were 5 language corrections for each scene, where 3 of the corrections were given by us (for  
332 consistency across subjects and to familiarize subjects with the types of corrections), and 2 of the corrections  
333 were given by the subjects. Subjects were free to provide any language corrections, after which both  
334 approaches would modify the trajectory. After each correction, subjects had to rank their agreement on  
335 how well each method modified the trajectory and which trajectory they preferred, if any. Specifically, for  
336 each method, they had to choose whether the deformed trajectory was 1) Completely wrong, 2) Somewhat  
337 wrong, 3) Neutral, 4) Somewhat correct, or 5) Completely correct. After each trajectory deformation and at  
338 the end of the experiments, they had to compare both methods. For this, they had to choose whether 1)

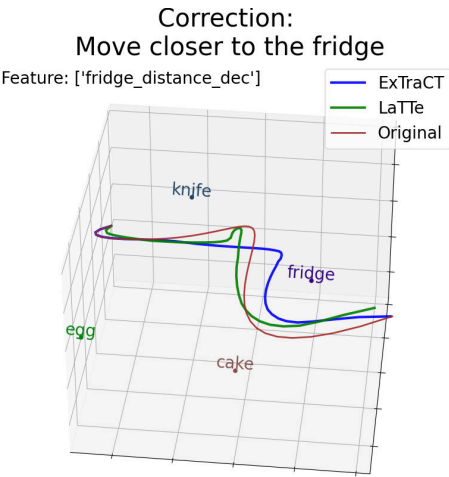
**Table 4.** Simulation User Study Results

| Approach | Overall       |                          | Object Distance Changes |                           | Cartesian Changes |                          |
|----------|---------------|--------------------------|-------------------------|---------------------------|-------------------|--------------------------|
|          | Accuracy      | Mean User Rank           | Accuracy                | Mean User Rank            | Accuracy          | Mean User Rank           |
| ExTraCT  | <b>88.00%</b> | <b>4.471</b> $\pm$ 0.062 | <b>96.53%</b>           | <b>4.521</b> $\pm$ 0.0668 | <b>100.00%</b>    | <b>4.776</b> $\pm$ 0.060 |
| LaTTe    | 55.56%        | 2.840 $\pm$ 0.0790       | 47.22%                  | 2.722 $\pm$ 0.0988        | 94.83%            | 3.207 $\pm$ 0.136        |

339 Method 1 was much better, 2) Method 1 was a bit better, 3) Both the methods were the same, 4) Method 2  
340 was a bit better, or 5) Method 2 was much better. At the end of the study, subjects were asked to elaborate  
341 on which method they preferred and why. We also measured the performance based on the accuracy of the  
342 trajectory deformations, as outlined in Section 4.2.1.



**Figure 5a.**



**Figure 5b.**

**Figure 5.** The deformed trajectories and features matched for (A) scene 2 and (B) for scene 3. Note that the modified trajectory by LaTTe opposes the corrections provided in these examples.

343 4.3.1 Results

344 Table 4 shows the results of our user study, showing better accuracy and preference for our method,  
345 supporting **H2** and **H3**. A Wilcoxon signed-rank test showed a significant difference in the rankings  
346 between our method and LaTTe ( $p < 0.0001$ ) on how well the trajectory followed the correction. When  
347 comparing the two methods after each correction, subjects rated that our method was better 85.5% of the  
348 cases, LaTTe was better 6.2% of the cases, and there was no difference for 8.0%. When asked at the end of  
349 the experiments, all the subjects preferred our method, with 33.3% of subjects rating that our method was  
350 “slightly better”, and 66.7% of subjects rating that our method was “much better” than LaTTe.

351 The higher preference for our method could be largely attributed to the accuracy of our approach, with  
352 many subjects stating that ExTraCT “follows [the] correction more”. LaTTe did not deform trajectories  
353 according to the corrections for many cases, such as in Fig. 5a and 5b, where the deformed trajectories  
354 were opposite to the corrections.



### 4.3.2 Failure Cases

There were 12.00% of failure cases with ExTraCT. Due to the explainability of our approach, we can analyse the feature matching and confidence score for each failure case (**H4**). We labelled the correct feature(s) for corrections within our feature space. For example, “Move below the fridge”, which contains a directional component for an object, is out of our feature space. These failures can be attributed to a lack of a correct deformation function. There were 12 such corrections (5.33%), which we removed from subsequent analysis.

The remaining 15 failures could be attributed to two reasons. Of these, 10 failures were due to multiple trajectory modification features within a single correction, such as “Move away from bottle and then closer to cup”. Since we assumed that each correction only contains one feature, such failures were expected. 5 failures were due to incorrect feature mapping, i.e., “Stay further away from the pineapple” was incorrectly mapped to `pineapple_distance_decrease`, as the most similar phrase was “Stay close to pineapple”; and “Move to the cake” was mapped to `cake_distance_increase` since the most similar phrase was “Move further away from cake”. These failures highlight the limitations of using Sentence Transformers for capturing semantic nuances in language corrections. Fine-tuning the model with in-domain data could improve the performance.

### 4.3.3 Effect of textual description templates on performance

We investigated the impact of the size of textual descriptions (number of phrases in each textual description) and phrase selection on performance. We selected phrases less frequently provided in the simulation experiments from the original set of textual description templates. The resulting textual description templates contained only 2 phrases, highlighted in bold in Table 1. Using this smaller set of phrases, we examined the features matched for the 59 unique language corrections from the simulation study. With a smaller set of language phrases, the number of errors in terms of feature mapping remained at 6, but there were 3 instances of low confidence. Even though the number of errors was the same, the phrases with incorrect feature mapping differed.

We also demonstrate how we can easily add phrases to our textual description templates and improve the system’s performance. We added the following phrases that were incorrectly mapped to the feature during the simulation study – “Stay further away from the {obj}” to `obj_distance_inc`; “Move towards {obj}”, “Move to {obj}” to `obj_distance_dec`; and “Move to the right” to `y_cart_inc`. After adding the phrases with incorrect feature mappings to the textual description templates, there were no more inaccurate feature mappings.

While pre-trained LLMs demonstrate promise in semantic similarity matching, their ability to generalize to the full diversity of language is influenced by the comprehensiveness and size of their training data. Using a larger pre-trained LLM could improve performance, but its computational demands can also increase significantly. Our approach’s explainability provides valuable insights to enhance the system’s performance. We demonstrate that expanding the phrases in the textual description templates can improve language grounding to features, improving our approach’s generalizability to language variations.

## 4.4 User Studies with Real Arm

We also conducted user studies using the xArm-6 robotic arm (UFactory, China) with the xArm two-finger gripper. An Intel Realsense Depth Camera D435 was mounted on the xArm-6 gripper to capture images of the scene to estimate the location of the objects. Additionally, for LaTTe, the bounding boxes of the objects

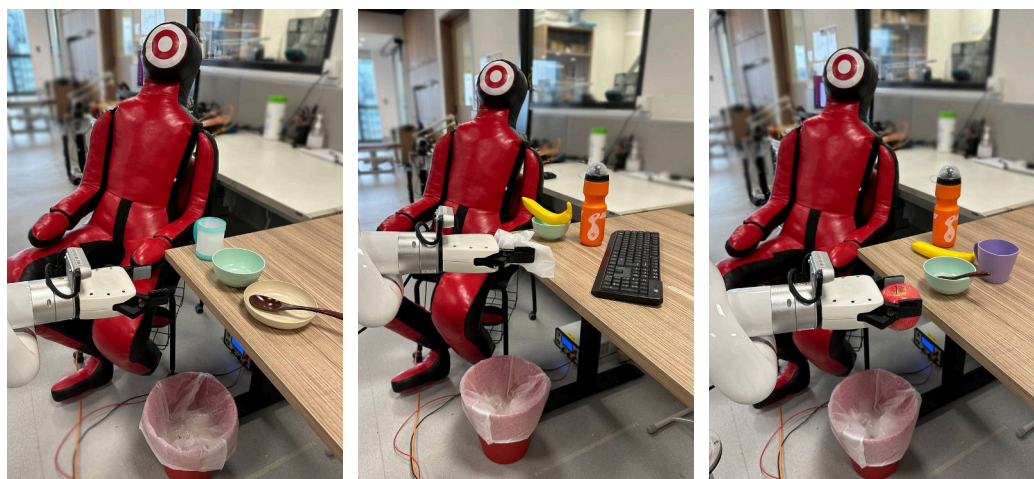


Figure 6a.

Figure 6b.

Figure 6c.

**Figure 6.** Setup of the experiments using the xArm-6 manipulator for (A) scene 1 (B) scene 2 and (C) scene 3. For safety, users were seated outside the robot's workspace and a dummy was placed at the scene.

were also estimated to obtain the CLIP embeddings of the objects. Object detection was performed using Mask R-CNN (He et al., 2017) in our experiments. TrajOpt (Schulman et al., 2014) was used to optimize the trajectory based on the robot kinematic constraints.

We recruited 5 subjects (2 male, 3 female) for a within-subject study, where each subject evaluated two methods – our approach (ExTraCT) and LaTTe.

#### 4.4.1 Setup and Protocol

There were 3 scenes with different objects on the tables. Fig. 6 shows the scenes used for real arm experiments. The object types, number of objects and object positions varied across the scenes. For the first scene, we provided the language corrections for consistency across subjects and also to familiarize the subjects with the types of corrections. We did not explicitly set a task for the robot to complete in Scene 1. The subjects had to rate the performance of the following corrections for both methods – "Go down", "Keep a bigger distance away from bowl", "Stay closer to spoon".

For the next 2 scenes, subjects had to use language corrections to modify a trajectory to complete tasks. Subjects were informed that they could only give 1 correction at a time, with 3 corrections required. The 2nd scene required subjects to throw trash into the bin while avoiding the food items and objects on the table. The 3rd scene required subjects to bring an apple to a dummy for handover while staying away from the bowl and staying closer to the table. In both scenes, subjects also had to modify the trajectory's final waypoint to move closer to the bin or the dummy. The language corrections were manually typed into the computer running both methods. Subjects completed both methods for each scene before moving to the next scene.

The order of the methods was randomized across subjects to counteract the effects of novelty and practice. Subjects filled out a qualitative survey after each task and each method, rating on a scale of 1 to 5 whether the modified trajectory followed the language correction. At the end of the study, subjects were asked to elaborate on their preferred method and why. We also measured the accuracy of the trajectory deformations using the same method as in Section 4.2.1.

4.4.2 Results

The real-arm study showed similar results to the simulation study (Table 5), where ExTraCT deformed trajectories more accurately than LaTTe (H2). A Wilcoxon signed-rank test was performed on the user rankings. Statistically significant differences were found, with our trajectory deformations following the corrections better ( $p < 0.0001$ ), showing support for H3. Subjects generally preferred our method, with 80% of subjects rating our method slightly better or much better than LaTTe. Subjects preferred our method because it “captures the user’s intention better” and has a “greater accuracy”.

Table 5. Real Arm Experiment Results

| Method  | Accuracy | Mean User Rank |
|---------|----------|----------------|
| ExTraCT | 97.78%   | 4.38 ± 0.12    |
| LaTTe   | 66.67%   | 3.16 ± 0.20    |

4.5 Application in Assistive Feeding

We deployed our framework in an assistive feeding task to show that our framework is versatile and can be applied across diverse scenarios. To tailor our framework for this context, we defined two features – *bite size* and *feeding speed*. The features and textual descriptions are shown in Table 6. We show some sample interactions on how the amount of food scooped can be modified using language in Fig. 7. The modularity of our framework allows different trajectory modification methods to be used depending on the task’s specific requirements. In this case, we used parameterized dynamic motion primitives (DMP) to modify the bite size.

Fig. 7b displays the variations in scooping trajectories based on language corrections. After the correction “Feed faster” was provided, the scooping speed increased. Similarly, following the correction “Feed me bigger bites”, the scooping trajectory was modified by scaling up the weight of the DMP, resulting in a larger bite size, as seen in Fig. 7d.

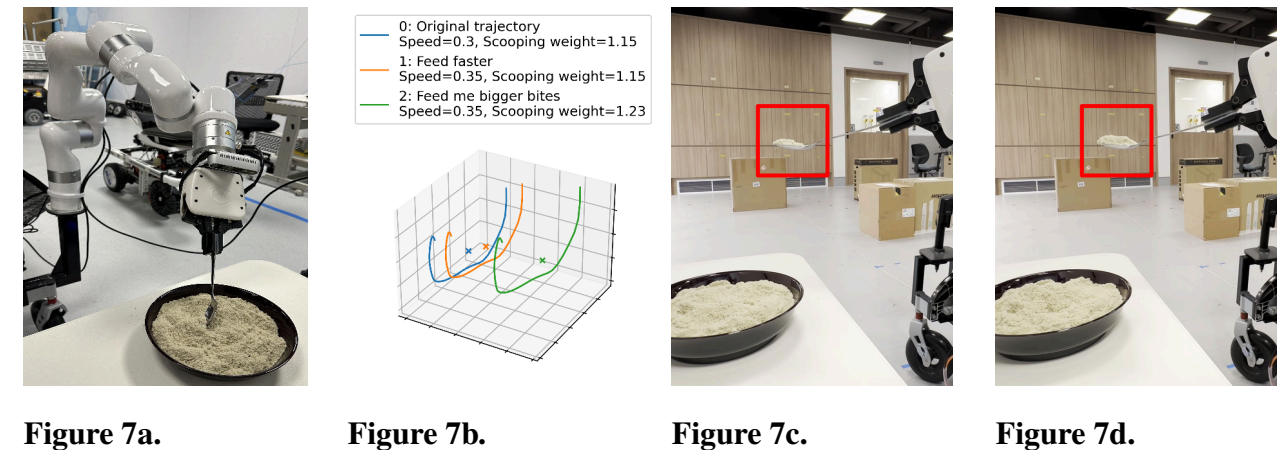


Figure 7. (A) Setup for assistive feeding using a spoon. (B) The change in scooping trajectories after a language correction is provided. (C) Original bite size. (D) Bite size after the correction “Feed me bigger bites” was provided.

**Table 6.** Features (F) and their Textual Descriptions (TD) for Assistive Feeding

|    |                                                                                                                           |                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| F  | bitesize_increase                                                                                                         | bitesize_decrease                                                                                                           |
| TD | I want a bigger bite<br>Increase the spoonful size<br>I want a larger bite next<br>I want more food<br>Increase bite size | I want a smaller bite<br>Decrease the spoonful size<br>I want a smaller bite next<br>I want less food<br>Decrease bite size |
| F  | speed_increase                                                                                                            | speed_decrease                                                                                                              |
| TD | Move faster<br>Increase speed<br>Too slow                                                                                 | Move slower<br>Decrease speed<br>Too fast                                                                                   |

## 5 CONCLUSION AND DISCUSSION

In conclusion, we propose ExTraCT, a modular trajectory correction framework. ExTraCT creates more accurate trajectory modification features for natural language corrections, which is important for safety and building trust in human-robot interaction. Our proposed architecture uses pre-trained LLMs for grounding user corrections and semantically maps them to the textual description of the features.

Our approach combines the strengths of hand-crafted features in trajectory deformations to generalize to different object configurations and initial trajectories and the language modelling capabilities of LLMs to handle language variations. By separating the problem of language understanding and trajectory modification, we have shown improvements in interpreting and executing language corrections, even for non-templated language phrases. The transparency of our approach allowed us to understand the root causes of failures, whether in language understanding or the trajectory deformation process, enabling more targeted improvements to the system.

This work is just a step towards understanding how explicitly obtaining the features for trajectory modification can help provide a more explainable and generalizable approach. In this work, our focus was on comparing our approach against an end-to-end method for language corrections with only a single feature. As such, our feature space is limited, and we cannot extract multiple features from a single correction. While our current implementation allows a chain of corrections, this approach may be inefficient for handling the natural complexity of language. Future work aims to increase our feature space and handle more complex language utterances, such as different intensities of trajectory modifications, compound sentences and referring expressions. Preliminary work has been done to identify the degree of change in language corrections by utilizing part-of-speech (POS) tagging and dependency tagging. These techniques effectively identify the modifiers in the language utterances, allowing us to modify the degree of change in the trajectory, as demonstrated in Figure 8. However, the intensity of modification will also be influenced by the scene context and initial trajectory. For example, consider the instruction, "Move slightly closer to the cup". The desired change in distance depends on the initial distance to the cup. If the cup is 10cm away, "slightly closer" might imply a 2cm adjustment, while a 30cm distance might imply a 10cm adjustment. To handle compound sentences, we believe decomposing them into simpler units offers a promising approach for a more generalizable solution. This decomposition can be achieved using standard natural language processing (NLP) methods or by leveraging frameworks like Frame Semantics (Fillmore et al., 2006; Vanzo et al., 2020) to identify multiple frames.

Another direction is to look into bi-directional communication between the robot and the user, allowing the robot to ask the user for clarifications in cases of uncertainty (i.e., low confidence in feature mapping), as has been demonstrated in Shridhar et al. (2020); Yow et al. (2024). This is important as we expand our feature space, as more features and textual descriptions could lead to cases where the same utterance could map to multiple features. The ability to integrate uncertainty handling in our framework also offers an additional advantage over end-to-end methods.

We hope our framework can provide a more transparent approach to learning human preferences, which can serve as a basis for transferring human preferences across different contexts.

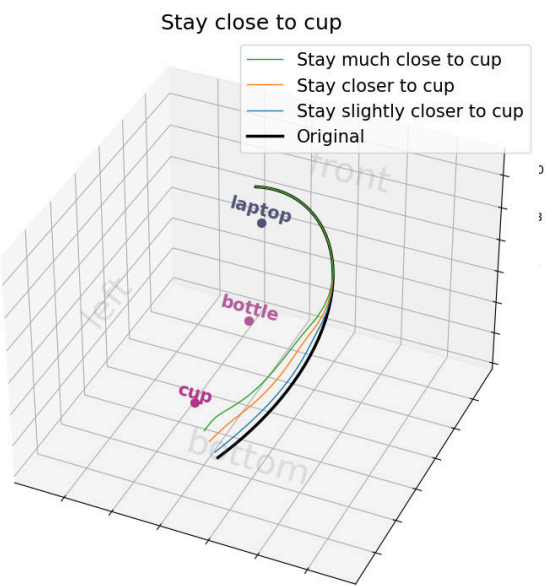


Figure 8a.

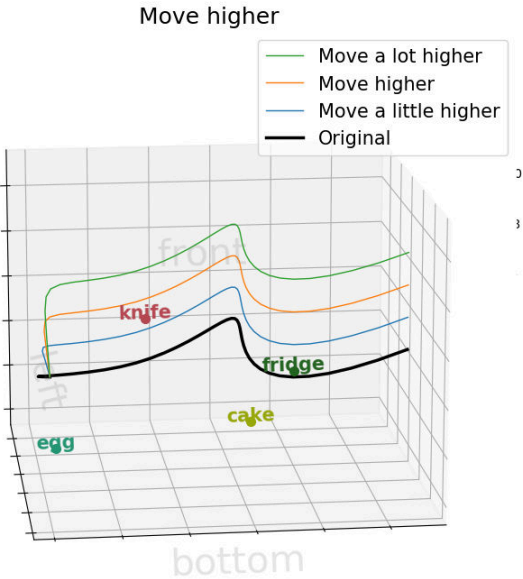


Figure 8b.

**Figure 8.** Trajectory modifications based on different intensities in the language utterance. (A) Different intensities for object distance changes. (B) Different intensities for cartesian changes.

ETHICS STATEMENT

The studies involving humans were approved by the Nanyang Technological University Institutional Review Board. The participants provided their written consent to participate in the studies.

CONFLICT OF INTEREST STATEMENT

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

AUTHOR CONTRIBUTIONS

JY: Writing - original draft, Conceptualization, Methodology, Formal analysis; NPG: Writing - review & editing, Conceptualization; MR: Writing - review & editing, Conceptualization; WTA: Conceptualization, Supervision, Funding acquisition



## FUNDING

The research was supported by the Rehabilitation Research Institute of Singapore and the National Research Foundation Singapore (NRF) under its Campus for Research Excellence and Technological Enterprise (CREATE) programme.

## ACKNOWLEDGMENTS

The research was conducted at the Future Health Technologies at the Singapore-ETH Centre, which was established collaboratively between ETH Zurich and the National Research Foundation Singapore.

The authors acknowledge using OpenAI's ChatGPT to edit the manuscript. They have carefully modified and verified the content generated by ChatGPT.

## REFERENCES

- Adadi, A. and Berrada, M. (2018). Peeking inside the black-box: a survey on explainable artificial intelligence (xai). *IEEE access* 6, 52138–52160
- Bajcsy, A., Losey, D. P., O'Malley, M. K., and Dragan, A. D. (2018). Learning from physical human corrections, one feature at a time. In *Proceedings of the 2018 ACM/IEEE International Conference on Human-Robot Interaction*. 141–149
- Bajcsy, A., Losey, D. P., O'malley, M. K., and Dragan, A. D. (2017). Learning robot objectives from physical human interaction. In *Conference on Robot Learning* (PMLR), 217–226
- Bobu, A., Wiggert, M., Tomlin, C., and Dragan, A. D. (2021). Feature expansive reward learning: Rethinking human input. In *Proceedings of the 2021 ACM/IEEE International Conference on Human-Robot Interaction*. 216–224
- Broad, A., Arkin, J., Ratliff, N., Howard, T., and Argall, B. (2017). Real-time natural language corrections for assistive robotic manipulators. *The International Journal of Robotics Research* 36, 684–698
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, eds. H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin (Curran Associates, Inc.), vol. 33, 1877–1901
- Bucker, A., Figueredo, L., Haddadin, S., Kapoor, A., Ma, S., Vemprala, S., et al. (2023). Latte: Language trajectory transformer. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 7287–7294. doi:10.1109/ICRA48891.2023.10161068
- Bucker, A., Figueredo, L., Haddadin, S., Kapoor, A., Ma, S., and Bonatti, R. (2022). Reshaping robot trajectories using natural language commands: A study of multi-modal data alignment using transformers. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (IEEE), 978–984
- [Dataset] Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al. (2022). Palm: Scaling language modeling with pathways. doi:10.48550/ARXIV.2204.02311
- Cui, Y., Karamcheti, S., Palleti, R., Shivakumar, N., Liang, P., and Sadigh, D. (2023). No, to the right: Online language corrections for robotic manipulation via shared autonomy (Association for Computing Machinery), HRI '23, 93–101. doi:10.1145/3568162.3578623
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*
- Fillmore, C. J. et al. (2006). Frame semantics. *Cognitive linguistics: Basic readings* 34, 373–400
- He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 2961–2969



- Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., and Fei-Fei, L. (2023). Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*
- Jain, A., Sharma, S., Joachims, T., and Saxena, A. (2015). Learning preferences for manipulation tasks from online coactive feedback. *The International Journal of Robotics Research* 34, 1296–1313
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., et al. (2023). Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)* (IEEE), 9493–9500
- Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., et al. (2023). Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*
- Minderer, M., Gritsenko, A., Stone, A., Neumann, M., Weissenborn, D., Dosovitskiy, A., et al. (2022). Simple open-vocabulary object detection. In *European Conference on Computer Vision* (Springer), 728–755
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., et al. (2021). Learning transferable visual models from natural language supervision. In *International conference on machine learning* (PMLR), 8748–8763
- Redmon, J. and Farhadi, A. (2018). Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*
- Reimers, N. and Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (Association for Computational Linguistics)
- Schulman, J., Duan, Y., Ho, J., Lee, A., Awwal, I., Bradlow, H., et al. (2014). Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research* 33, 1251–1270
- Sharma, P., Sundaralingam, B., Blukis, V., Paxton, C., Hermans, T., Torralba, A., et al. (2022). Correcting robot plans with natural language feedback. *arXiv preprint arXiv:2204.05186*
- Shridhar, M., Mittal, D., and Hsu, D. (2020). Ingress: Interactive visual grounding of referring expressions. *The International Journal of Robotics Research* 39, 217–232
- Spencer, J., Choudhury, S., Barnes, M., Schmittle, M., Chiang, M., Ramadge, P., et al. (2020). Learning from interventions: Human-robot interaction as both explicit and implicit feedback. In *16th Robotics: Science and Systems, RSS 2020* (MIT Press Journals)
- Tellex, S., Gopalan, N., Kress-Gazit, H., and Matuszek, C. (2020). Robots that use language. *Annual Review of Control, Robotics, and Autonomous Systems* 3, 25–55
- Vanzo, A., Croce, D., Bastianelli, E., Basili, R., and Nardi, D. (2020). Grounded language interpretation of robotic commands through structured learning. *Artificial Intelligence* 278, 103181
- [Dataset] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., and Zhou, M. (2020). Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers
- Yow, J.-A., Garg, N. P., and Ang, W. T. (2024). Shared autonomy of a robotic manipulator for grasping under human intent uncertainty using pomdps. *IEEE Transactions on Robotics* 40, 332–350. doi:10.1109/TRO.2023.3334631
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., et al. (2023). Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*
- Zha, L., Cui, Y., Lin, L.-H., Kwon, M., Arenas, M. G., Zeng, A., et al. (2023). Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. In *2nd Workshop on Language and Robot Learning: Language as Grounding*